CHAPTER 1

INTRODUCTION

CHAPTER OVERVIEW

This chapter gives an idea of natural language processing (NLP) and information retrieval (IR). Various levels of analysis involved in NLP along with the knowledge used by these levels of analysis are discussed. Some of the difficulties in analysing text and specific factors that make automatic processing of languages difficult are also touched upon. The chapter underlines the role of grammar in language processing and introduces transformational grammar. Indian languages differ a lot from English. These differences are clearly pointed out. Further, a number of NLP applications are introduced along with some of the early NLP systems. Towards the end, information retrieval is discussed.

1.1 WHAT IS NATURAL LANGUAGE PROCESSING (NLP)

Language is the primary means of communication used by humans. It is the tool we use to express the greater part of our ideas and emotions. It shapes thought, has a structure, and carries meaning. Learning new concepts and expressing ideas through them is so natural that we hardly realize how we process natural language. But there must be some kind of representation in our mind, of the content of language. When we want to express a thought, this content helps represent language in real time. As children, we never learn a computational model of language, yet this is the first step in the automatic processing of languages. *Natural language processing* (NLP) is concerned with the development of computational models of aspects of human language processing. There are two main reasons for such development:

1. To develop automated tools for language processing
2. To gain a better understanding of human communication

Building computational models with human language-processing abilities requires a knowledge of how humans acquire, store, and process language. It also requires a knowledge of the world and of language.

implications. We are here considering the text from of the language and the content of it as knowledge.

Language, being a medium of expression, is the outer form of the content it expresses. The same content can be expressed in different languages. But can language be separated from its content? If so, how can the content itself be represented? Generally, the meaning of one language is written in the same language (but with a different set of words). It may also be written in some other, formal, language. Hence, to process a language means to process the content of it. As computers are not able to understand natural language, methods are developed to map its content in a formal language. Sometimes, formal language content may have to be expressed in a natural language as well. Thus, in this book, language is taken up as a knowledge representation tool that has historically represented the whole body of knowledge and that has been modified, maybe through generation of new words, to include new ideas and situations. The language and speech community, on the other hand, considers a language as a set of sounds that, through combinations, conveys meaning to a listener. However, we are concerned with representing and processing text only. Language (text) processing has different levels, each involving different types of knowledge. We now discuss various levels of processing and the types of knowledge it involves.

The simplest level of analysis is *lexical analysis*, which involves analysis of words. Words are the most fundamental unit (syntactic as well as semantic) of any natural language text. Word-level processing requires morphological knowledge, i.e., knowledge about the structure and formation of words from basic units (morphemes). The rules for forming words from morphemes are language specific.

The next level of analysis is *syntactic analysis*, which considers a sequence of words as a unit, usually a sentence, and finds its structure. Syntactic analysis decomposes a sentence into its constituents (or words) and identifies how they relate to each other. It captures grammaticality or non-grammaticality of sentences by looking at constraints like word order, number, and case agreement. This level of processing requires syntactic knowledge, i.e., knowledge about how words are combined to form larger units such as phrases and sentences, and what constraints are imposed on them. Not every sequence of words results in a sentence. For example, 'I went to the market' is a valid sentence whereas 'went the I market to' is not. Similarly, 'She is going to the market ' is valid, but 'She are going to the market' is not. Thus, this level of analysis requires detailed knowledge about rules of grammar.

Yet another level of analysis is *semantic analysis*. Semantics is associated with the meaning of the language. Semantic analysis is concerned with creating meaningful representation of linguistic inputs. The general idea of semantic interpretation is to take natural language sentences or utterances and map them onto some representation of meaning. Defining meaning components is difficult as grammatically valid sentences can be meaningless. One of the famous examples is, 'Colorless green ideas sleep furiously' (Chomsky 1957). The sentence is well-formed, i.e., syntactically correct, but semantically anomalous. However, this does not mean that syntax has no role to play in meaning. Bach (2002) considers:

> *'... semantics to be a projection of its syntax. That is semantic structure is interpreted syntactic structure.'*

But definitely, syntax is not the only component to contribute meaning. Our conception of meaning is quite broad. We feel that humans apply all sorts of knowledge (i.e., lexical, syntactic, semantic, discourse, pragmatic, and world knowledge) to arrive at the meaning of a sentence. The starting point in semantic analysis, however, has been lexical semantics (meaning of words). A word can have a number of possible meanings associated with it. But in a given context, only one of these meanings participates. Finding out the correct meaning of a particular use of word is necessary to find meaning of larger units. However, the meaning of a sentence cannot be composed solely on the basis of the meaning of its words. Consider the following sentences:

Kabir and Ayan are married. (1.1a)

Kabir and Suha are married. (1.1b)

Both sentences have identical structures, and the meanings of individual words are clear. But most of us end up with two different interpretations. We may interpret the second sentence to mean that Kabir and Suha are married to each other, but this interpretation does not occur for the first sentence. Syntactic structure and compositional semantics fail to explain these interpretations. We make use of pragmatic information. This means that semantic analysis requires pragmatic knowledge besides semantic and syntactic knowledge.

A still higher level of analysis is *discourse analysis*. Discourse-level processing attempts to interpret the structure and meaning of even larger units, e.g., at the paragraph and document level, in terms of words, phrases, clusters, and sentences. It requires the resolution of anaphoric references and identification of discourse structure. It also requires discourse knowledge, that is, knowledge of how the meaning of a sentence is determined by preceding sentences—e.g., how a pronoun refers to the

preceding noun—and how to determine the function of a sentence in the text. In fact, pragmatic knowledge may be needed for resolving anaphoric references. For example, in the following sentences, resolving the anaphoric reference 'they' requires pragmatic knowledge:

The district administration refused to give the trade union permission for the meeting because they feared violence. (1.2a)

The district administration refused to give the trade union permission for the meeting because they oppose government. (1.2b)

The highest level of processing is *pragmatic analysis*, which deals with the purposeful use of sentences in situations. It requires knowledge of the world, i.e., knowledge that extends beyond the contents of the text. The Cyc project (Lenat 1986) at University of Austin is an attempt to utilize world knowledge in NLP. However, its usefulness in a general-domain NLP system is yet to be demonstrated. Furthermore, whether or not semantics can be associated with a symbol manipulator and whether humans use logic in the same way as the Cyc project, are both issues of debate.

1.4 THE CHALLENGES OF NLP

There are a number of factors that make NLP difficult. These relate to the problems of representation and interpretation. Language computing requires precise representation of content. Given that natural languages are highly ambiguous and vague, achieving such representation can be difficult. The inability to capture all the required knowledge is another source of difficulty. It is almost impossible to embody all sources of knowledge that humans use to process language. Even if this were done, it is not possible to write procedures that imitate language processing as done by humans. In this section, we detail some of the problems associated with NLP.

Perhaps the greatest source of difficulty in natural language is identifying its semantics. The principle of compositional semantics considers the meaning of a sentence to be a composition of the meaning of words appearing in it. In the earlier section, we saw a number of examples where this principle failed to work. Our viewpoint is that words alone do not make a sentence. Instead, it is the words as well as their syntactic and semantic relation that give meaning to a sentence. As pointed out by Wittgenstein (1953): 'The meaning of a word is its use in the language.' A language keeps on evolving. New words are added continually and existing

words are introduced in new context. For example, most newspapers and TV channels use 9/11 to refer to the terrorist act on the World Trade Centre in the USA in 2004. When we process written text or spoken utterances, we have access to underlying mental representation. The only way a machine can learn the meaning of a specific word in a message is by considering its context, unless some explicitly coded general world or domain knowledge is available. The context of a word is defined by co-occurring words. It includes everything that occurs before or after a word. The frequency of a word being used in a particular sense also affects its meaning. The English word 'while' was initially used to mean 'a short interval of time'. But now it is more in use as a conjunction. None of the usages of 'while' discussed in this chapter correspond to this meaning.

Idioms, metaphor, and ellipses add more complexity to identify the meaning of the written text. As an example, consider the sentence:

The old man finally kicked the bucket. (1.3)

The meaning of this sentence has nothing to do with the words 'kick' and 'bucket' appearing in it.

Quantifier-scoping is another problem. The scope of quantifiers (the, each, etc.) is often not clear and poses problem in automatic processing.

The ambiguity of natural languages is another difficulty. These go unnoticed most of the times, yet are correctly interpreted. This is possible because we use explicit as well as implicit sources of knowledge. Communication via language involves two brains not just one—the brain of the speaker/writer and that of the hearer/reader. Anything that is assumed to be known to the receiver is not explicitly encoded. The receiver possesses the necessary knowledge and fills in the gaps while making an interpretation. As humans, we are aware of the context and current cultural knowledge, and also of the language and traditions, and utilize these to process the meaning. However, incorporating contextual and world knowledge poses the greatest difficulty in language computing. An example of cultural impact on language is the representation of different shades of white in the Eskimo world. It may be hard for a person living in plain to distinguish among various shades. Similarly, to an Indian, the word 'Taj' may mean a monument, a brand of tea, or a hotel, which may not be so for a non-Indian. Let us now take a look at the various sources of ambiguities in natural languages.

The first level of ambiguity arises at the word level. Without much effort, we can identify words that have multiple meanings associated with

them, e.g., bank, can, bat, and still. A word may be ambiguous in its part-of-speech or it may be ambiguous in its meaning. The word 'can' is ambiguous in its part-of-speech whereas the word 'bat' is ambiguous in its meaning. We hardly consider all possible meanings of a word to get the correct one. A program on the other hand, must be explicitly coded to resolve each meaning. Hence, we need to develop various models and algorithms to resolve them. Deciding whether 'can' is a noun or a verb is solved by 'part-of-speech tagging' whereas identifying whether a particular use of 'bank' corresponds to 'financial institution' sense or 'river bank' sense is solved by 'word sense disambiguation'. 'Part-of-speech tagging' and 'word sense disambiguation' algorithms are discussed in Chapters 3 and 5 respectively.

A sentence may be ambiguous even if the words are not, for example, the sentence: '*Stolen rifle found by tree.*' None of the words in this sentence is ambiguous but the sentence is. This is an example of structural ambiguity. Verb sub-categorization may help to resolve this type of ambiguity but not always. Probabilistic parsing, which is discussed in Chapter 4, is another solution. At a still higher level are pragmatic and discourse ambiguities. Ambiguities are discussed in Chapter 5.

A number of grammars have been proposed to describe the structure of sentences. However, there are an infinite number of ways to generate them, which makes writing grammar rules, and grammar itself, extremely complex. On top of it, we often make correct semantic interpretations of non-grammatical sentences. This fact makes it almost impossible for grammar to capture the structure of all and only meaningful text.

1.5 LANGUAGE AND GRAMMAR

Automatic processing of language requires the rules and exceptions of a language to be explained to the computer. Grammar defines language. It consists of a set of rules that allows us to parse and generate sentences in a language. Thus, it provides the means to specify natural language. These rules relate information to coding devices at the language level—not at the world-knowledge level (Bharati et al. 1995). However, since world knowledge affects both the coding (i.e., words) and the coding convention (structure), this blurs the boundary between syntax and semantics. Nevertheless such a separation is made because of the ease of processing and grammar writing.

The main hurdle in language specification comes from the constantly changing nature of natural languages and the presence of a large number

of hard-to-specify exceptions. Several efforts have been made to provide such specifications, which has led to the development of a number of grammars. Main among them are transformational grammar (Chomsky 1957), lexical functional grammar (Kaplan and Bresnan 1982), government and binding (Chomsky 1981), generalized phrase structure grammar, transformational grammar (Chomsky 1957), dependency grammar, Paninian grammar, and tree-adjoining grammar (Joshi 1985). Some of these grammars focus on derivation (e.g., phrase structure grammar) while others focus on relationships (e.g., dependency grammar, lexical functional grammar, Paninian grammar, and link grammar). We discuss some of these in Chapter 2. The greatest contribution to grammar comes from Noam Chomsky, who proposed a hierarchy of formal grammar based on level of complexity. These grammars use phrase structure rules (or rewrite rules). The term 'generative grammar' is often used to refer to the general framework introduced by Chomsky. Generative grammar basically refers to any grammar that uses a set of rules to specify or generate all and only grammatical (well-formed) sentences in a language. Chomsky argued that phrase structure grammars are not adequate to specify natural language. He proposed a complex system of transformational grammar in his book on *Syntactic Structures* (1957), in which he suggested that each sentence in a language has two levels of representation, namely, a deep structure and a surface structure (See Figure 1.1). The mapping from deep structure to surface structure is carried out by transformations. In the following paragraphs, we introduce transformational grammar.

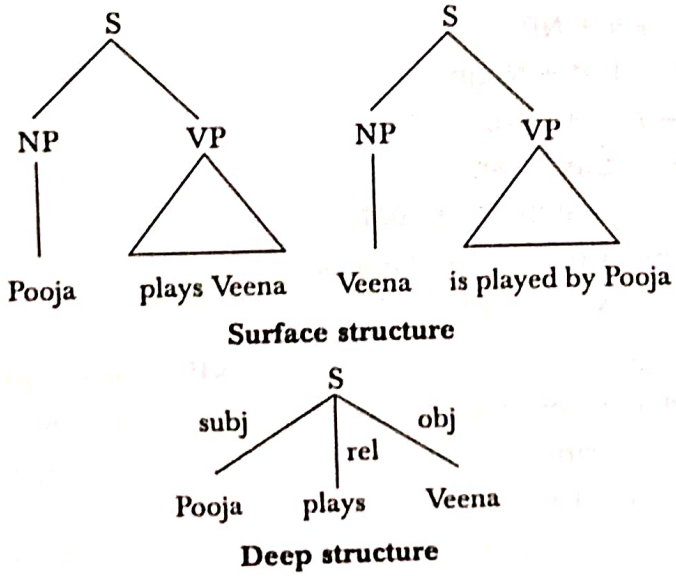

Figure 1.1 Surface and deep structures of sentence

Transformational grammar was introduced by Chomsky in 1957. Chomsky argued that an utterance is the surface representation of a 'deeper structure' representing its meaning. The deep structure can be transformed in a number of ways to yield many different surface-level representations. Sentences with different surface-level representations having the same meaning, share a common deep-level representation. Chomsky's theory was able to explain why sentences like

Pooja plays veena. (1.4a)

Veena is played by Pooja. (1.4b)

have the same meaning, despite having different surface structures (roles of subject and object are inverted). Both the sentences are being generated from the same 'deep structure' in which the deep subject is Pooja and the deep object is the veena.

Transformational grammar has three components:

1. Phrase structure grammar
2. Transformational rules
3. Morphophonemic rules—These rules match each sentence representation to a string of phonemes.

Each of these components consists of a set of rules. Phrase structure grammar consists of rules that generate natural language sentences and assign a structural description to them. As an example, consider the following set of rules:

S → NP + VP
VP → V + NP
NP → Det + Noun
V → Aux + Verb
Det → the, a, an, ...
Verb → catch, write, eat, ...
Noun → police, snatcher, ...
Aux → will, is, can, ...

In these rules, S stands for sentence, NP for noun phrase, VP for verb phrase, and Det for determiner. Sentences that can be generated using these rules are termed grammatical. The structure assigned by the grammar is a constituent structure analysis of the sentence.

The second component of transformational grammar is a set of transformation rules, which transform one phrase-maker (underlying) into another phrase-marker (derived). These rules are applied on the terminal

string generated by phrase structure rules. Unlike phrase structure rules, transformational rules are heterogeneous and may have more than one symbol on their left hand side. These rules are used to transform one surface representation into another, e.g., an active sentence into passive one. The rule relating active and passive sentences (as given by Chomsky) is

$$NP_1 - Aux - V - NP_2 \rightarrow NP_2 - Aux + be + en - V - by + NP_1$$

This rule says that an underlying input having the structure NP–Aux–V–NP can be transformed to NP – Aux + be + en – V – by + NP. This transformation involves addition of strings 'be' and 'en' and certain re-arrangements of the constituents of a sentence. Transformational rules can be obligatory or optional. An obligatory transformation is one that ensures agreement in number of subject and verb, etc., whereas an optional transformation is one that modifies the structure of a sentence while preserving its meaning. Morphophonemic rules match each sentence representation to a string of phonemes.

Consider the active sentence:

The police will catch the snatcher. (1.5)

The application of phrase structure rules will assign the structure shown in Figure 1.2 to this sentence.

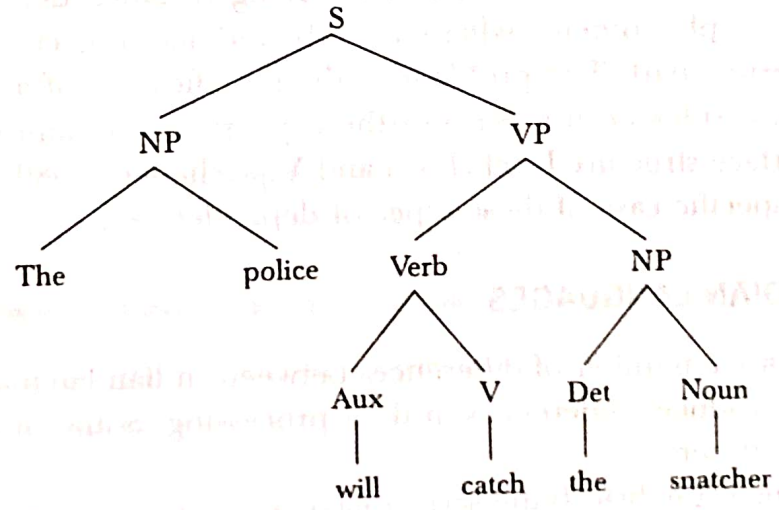

Figure 1.2 Parse structure of sentence (1.5)

The passive transformation rules will convert the sentence into:
The + culprit + will + be + en + catch + by + police (Figure 1.3).

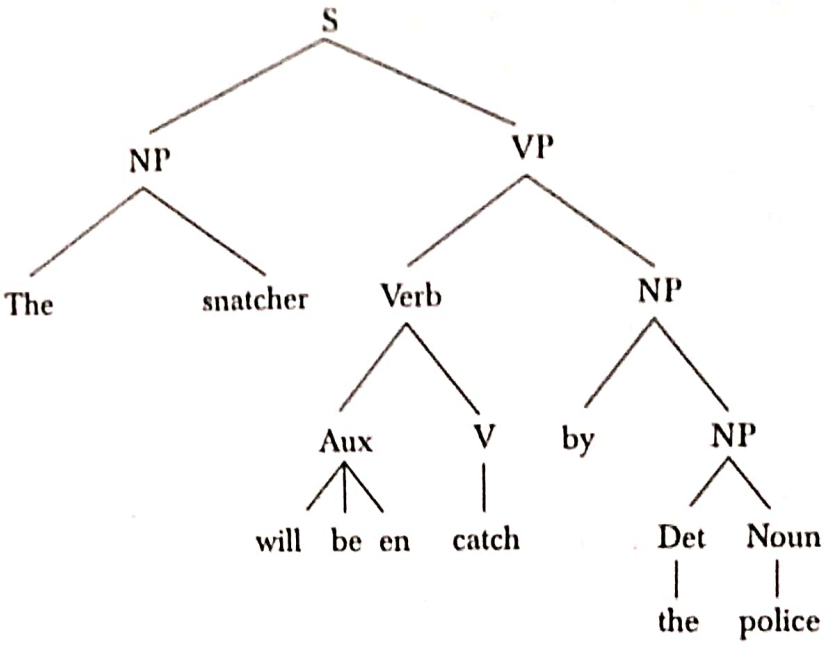

Figure 1.3 Structure of sentence (1.5) after applying passive transformations

Another transformational rule will then reorder 'en + catch' to 'catch + en' and subsequently one of the morphophonemic rules will convert 'catch + en' to 'caught'. In general, the noun phrase is not always as simple as in sentence (1.5). It may contain other embedded structures, such as adjectives, modifiers, relative clause, etc. Long distance dependencies are other language phenomena that cannot be adequately handled by phrase structure rules. Long distance dependency refers to syntactic phenomena where a verb and its subject or object can be arbitrarily apart. The problem in the specification of appropriate phrase structure rules occurs because these phenomena cannot be localized at the surface structure level (Joshi and Vijayshanker 1989). Wh-movement[1] are a specific case of these types of dependencies.

1.6 PROCESSING INDIAN LANGUAGES

There are a number of differences between Indian languages and English. This introduces differences in their processing. Some of these differences are listed here.

- Unlike English, Indic scripts have a non-linear structure.
- Unlike English, Indian languages have SOV (Subject-Object-Verb) as the default sentence structure.

[1]It refers to a syntactic phenomenon in which interrogative words, called wh-words, appear at the beginning of the sentence. For example, when the direct object of the verb 'read' in the sentence 'She is reading a book' is replaced with a wh-word, the sentence becomes 'What is she reading?' instead of 'She is reading what?'.

- Indian languages have a free word order, i.e., words can be moved freely within a sentence without changing the meaning of the sentence.
- Spelling standardization is more subtle in Hindi than in English.
- Indian languages have a relatively rich set of morphological variants.
- Indian languages make extensive and productive use of complex predicates (CPs).
- Indian languages use post-position (*Karakas*) case markers instead of prepositions.
- Indian languages use verb complexes consisting of sequences of verbs, e.g., गा रहा है (*ga raha hai*—singing) and खेल रही है (*khel rahi hai*—playing). The auxiliary verbs in this sequence provide information about tense, aspect, modality, etc.

Except for the direction in which its script is written, Urdu is closely related to Hindi. Both share similar phonology, morphology, and syntax. Both are free-word-order languages and use post-positions. They also share a large amount of their vocabulary. Differences in the vocabulary arise mainly because a significant portion of Urdu vocabulary comes from Persian and Arabic, while Hindi borrows much of its vocabulary from Sanskrit.

Paninian grammar provides a framework for Indian language models. These can be used for computation of Indian languages. The grammar focuses on extraction of Karaka relations from a sentence. We talk about the details of modelling in Chapter 2. A parsing framework based on Paninian grammar is introduced in Chapter 4 and issues involved in Indian language translation (using Paninian grammar theory) are discussed in Chapter 8.

1.7 NLP APPLICATIONS

Machine translation is the first application area of NLP. It involves the complete linguistic analysis of a natural language sentence, and linguistic generation of an output sentence. It is one of the most comprehensive and most challenging tasks in the area (AI-complete). However, the recent dramatic progress in the field of NLP has found interesting applications in information retrieval, information extraction, text summarization, etc. This book offers an extensive coverage of these recent applications, and also of traditional ones like machine translation and natural language generation. The focus has been on bridging the gap between theory and practice rather than on offering a gamut of linguistic, psychological, and computational theories.

The applications utilizing NLP include the following.

Machine Translation

This refers to automatic translation of text from one human language to another. In order to carry out this translation, it is necessary to have an understanding of words and phrases, grammars of the two languages involved, semantics of the languages, and world knowledge.

Speech Recognition

This is the process of mapping acoustic speech signals to a set of words. The difficulties arise due to wide variations in the pronunciation of words, homonym (e.g. dear and deer) and acoustic ambiguities (e.g., in the rest and interest).

Speech Synthesis

Speech synthesis refers to automatic production of speech (utterance of natural language sentences). Such systems can read out your mails on telephone, or even read out a storybook for you. In order to generate utterances, text has to be processed. So, NLP remains an important component of any speech synthesis system.

Natural Language Interfaces to Databases

Natural language interfaces allow querying a structured database using natural language sentences.

Information Retrieval

This is concerned with identifying documents relevant to a user's query. NLP techniques have found useful applications in information retrieval. Indexing (stop word elimination, stemming, phrase extraction, etc.), word sense disambiguation, query modification, and knowledge bases have also been used in IR system to enhance performance, e.g., by providing methods for query expansion. WordNet, LDOCE (*Longman Dictionary of Contemporary English*) and *Roget's Thesaurus* are some of the useful lexical resources for IR research.

Information Extraction

An information extraction system captures and outputs factual information contained within a document. Similar to an information retrieval system, it responds to a user's information need. However, unlike in an information retrieval system, the information need is not expressed as a keyword query. Instead it is specified as pre-defined database schemas or templates. Whereas an IR system identifies a subset of documents in a large repository of text database—e.g., in a library scenario, a subset of resources in a

library—an information extraction system identifies a subset of information within a document that fits the pre-defined template.

Question Answering

Given a question and a set of documents, a question answering system attempts to find the precise answer, or at least the precise portion of text in which the answer appears. This is unlike an IR system, which returns whole document that seems relevant to the user's query. A question answering system is different from an information extraction system in that the content that is to be extracted is unknown. In general, a question answering system benefits from having an information extraction system to identify entities in the text. A question answering system requires more NLP than an information retrieval system or an information extraction system. It requires not only precise analysis of questions and portions of texts but also semantic as well as background knowledge to answer certain type of questions.

Text Summarization

This deals with the creation of summaries of documents and involves syntactic, semantic, and discourse level processing of text.

1.8 SOME SUCCESSFUL EARLY NLP SYSTEMS

ELIZA (Weizenbaum 1966)

ELIZA is one of the earliest natural language understanding programs. It uses syntactic patterns to mimic human conversation with the user. Here is a sample conversation.

Eliza: Hello. I am ELIZA. How may I help you?
User: I am feeling a little bit sleepy.
Eliza: How long have you been feeling a little bit sleepy?
User: For almost half an hour.
Eliza: Please go on.

SysTran (System Translation)

The first SysTran machine translation system was developed in 1969 for Russian–English translation. SysTran also provided the first on-line machine translation service called Babel Fish, which is used by AltaVista search engines for handling translation requests from users.

TAUM METEO

This is a natural language generation system used in Canada to generate weather reports. It accepts daily weather data and generates weather reports in English and French.

***SHRDLU* (Winograd 1972)**

This is a natural language understanding system that simulates actions of a robot in a block world domain. It uses syntactic parsing and semantic reasoning to understand instructions. The user can ask the robot to manipulate the blocks, to tell the blocks configurations, and to explain its reasoning.

***LUNAR* (Woods 1977)**

This was an early question answering system that answered questions about moon rock.

1.9 INFORMATION RETRIEVAL

The availability of a large amount of text in electronic form has made it extremely difficult to get relevant information. Information retrieval systems aim at providing a solution to this.

The term 'information' should not be confused with the term 'entropy' (numerical measure of the uncertainty of an outcome) as it is used in communication theory. Information is being used here to reflect 'subject matter' or the 'content' of some text. We are not interested in 'digital communication', where bits and bytes are the information carriers. Instead our focus in on the communication taking place between human beings as expressed through natural languages. Information is always associated with some data (text, number, image, and so on): we are concerned with text only. Hence, we consider words as the carriers of information and written text as the message encoded in natural language.

As a cognitive activity, the word 'retrieval' refers to operation of accessing information from memory. We use the word 'retrieval' to refer to the operation of accessing information from some computer-based representation. Retrieval of information thus requires information to be processed and stored. Not all the information represented in computable form is retrieved. Instead, only the information relevant to the needs expressed in the form of query is located. In order to get this relevance, the stored and processed information needs to be compared against query representation. Information retrieval (IR) deals with all these facets. It is concerned with the organization, storage, retrieval, and evaluation of information relevant to the query.

Information retrieval deals with unstructured data. The retrieval is performed based on the content of the document rather than on its structure. The IR systems usually return a ranked list of documents. The IR components have been traditionally incorporated into different types

CHAPTER 3

WORD LEVEL ANALYSIS

CHAPTER OVERVIEW

This chapter focuses on processing carried out at word level, including methods for characterizing word sequences, identifying morphological variants, detecting and correcting misspelled words, and identifying correct part-of-speech of a word. The part-of-speech tagging methods covered in this chapter are: rule-based (linguistic), Stochastic (data-driven), and hybrid.

3.1 INTRODUCTION

As discussed in Chapter 1, natural language processing (NLP) involves different levels and complexities of processing. One way to analyse natural language text is by breaking it down into constituent units (words, phrases, sentences, and paragraphs) and then analyse these units. In Chapter 2, we discussed various language models that are used for analysing the syntax of natural language sentences. Before analysing syntax, we need to understand words, as words are the fundamental unit (syntactic as well as semantic) of any natural language text. This chapter focuses on NLP carried out at word level, including characterizing word sequences, identifying morphological variants, detecting and correcting misspelled words, and identifying correct part-of-speech of a word.

Regular expressions are a beautiful means for describing words. In many text applications, we wish to work with string patterns. Suppose you have just come across the word 'supernova'. It catches your interest and you jump to a search engine to find out more on 'supernovas'. But you do not know whether to type in 'supernova', 'Supernova', or 'supernovas'. Obviously, you need a system, which will retrieve relevant articles using any one of these word forms. Regular expressions are used for describing text strings in situations like this and in other information

retrieval applications. In this chapter, we introduce regular expressio and discuss standard notations for describing text patterns.

After defining regular expressions, we discuss their implementati using finite-state automaton (FSA). Readers who have gone through course in formal language theory will be familiar with FSA. The FS and their variants, such as finite state transducers, have found usef applications in speech recognition and synthesis, spell checking, an information extraction. As we will be using FSA throughout this boo we formally define FSAs in this chapter.

Errors in typing and spelling are quite common in text processin applications. Detecting and correcting these errors are the next topics o discussion in this chapter. Numerous web pages also contain misspelle words and often, query terms entered into the search engines ar misspelled. An interactive spelling facility that informs users of such error and presents appropriate corrections to them, is useful in these applications.

There are different classes of words. A word has many forms and the same word may have many different meanings depending on the context. Identifying the class to which a word belongs, its basic form, and its meaning are crucial to analysing text. This is the last topic covered in this chapter.

3.2 REGULAR EXPRESSIONS

Regular expressions, or regexes for short, are a pattern-matching standard for string parsing and replacement. They are a powerful way to find and replace strings that take a defined format. For example, regular expressions can be used to parse dates, urls and email addresses, log files, configuration files, command line switches, or programming scripts. They are useful tools for the design of language compilers and have been used in NLP for tokenization, describing lexicons, morphological analysis, etc. We have all used simplified forms of regular expressions, such as the file search patterns used by MS DOS, e.g., dir*.txt.

The use of regular expressions in computer science was made popular by a Unix-based editor, 'ed'. Perl was the first language that provided integrated support for regular expressions. It used a slash around each regular expression; we will follow the same notation in this book. However, slashes are not a part of regular expressions.

Regular expressions were originally studied as a part of theory of computation. They were first introduced by Kleene (1956). A regular expression is an algebraic formula whose value is a pattern consisting of a

set of strings, called the language of the expression. The simplest kind of regular expression contains a single symbol. For example, the expression

/a/

denotes the set containing the string 'a'. A regular expression may specify a sequence of characters also. For example, the expression

/supernova/

denotes the set that contains the string 'supernova' and nothing else. In a search application, the first instance of each match to regular expression is underlined in Table 3.1.

Table 3.1 Some simple regular expressions

Regular expression	Example patterns
/book/	The world is a book, and those who do not travel read only one page.
/book/	Reporters, who do not read the stylebook, should not criticize their editors.
/face/	Not everything that is faced can be changed. But nothing can be changed until it is faced.
/a/	Reason, Observation, and Experience—the Holy Trinity of Science.

3.2.1 Character Classes

Characters are grouped by putting them between square brackets. This way, any character in the class will match one character in the input. For example, the pattern /[abcd]/ will match (any of) a, b, c, and d. The use of brackets specifies a disjunction of characters. The regular expression /[0123456789]/ specifies any single digit. The character classes are important building blocks in expressions. They sometimes lead to cumbersome notation. For example, it is inconvenient to write the regular expression

/[abcdefghijklmnopqrstuvwxyz]/

to specify 'any lowercase letter'. In these cases, a dash is used to specify a range. The regular expression /[5–9]/ specifies any one of the characters 5, 6, 7, 8, or 9. The pattern /[m–p]/ specifies any one of the letter m, n, o, or p.

Regular expressions can also specify what a single character cannot be, by the use of a caret at the beginning. For example, the pattern /[^x]/ matches any single character except x. This interpretation is true only when a caret appears as the first symbol. If it occurs at any other place, it refers to the caret symbol itself. Table 3.2 shows a few examples explaining these concepts.

Table 3.2 Use of square brackets

RE	Match	Example patterns matched
[abc]	Match any of a, b, and c	'Refresher course will start tomorrow'
[A–Z]	Match any character between A and Z (ASCII order)	'the course will end on Jan. 10, 2006'
[^A–Z]	Match any character other than an uppercase letter	'TREC Conference'
[^abc]	Match anything other than a, b, and c	'TREC Conference'
[+*?.]	Match any of +, *, ?, or the dot.	'3 ±2 = 5'
[a^]	Match a or ^	'^ has three different uses.'

Regular expressions are *case sensitive*. The pattern /s/ matches a lower case 's' but not an uppercase 'S'. This means that the pattern /sana/ will not match the string /Sana/. This problem can be solved by using the disjunction of character s and S. The pattern /[sS]/ will match the strings containing either 's' or 'S'. The pattern /[sS]ana/ matches with the string 'sana' or 'Sana'.

While the use of square brackets solves the capitalization problem, we still need a solution for how to specify both 'supernova' and 'supernovas'. The pattern /[sS]upernova[sS]/ matches with any of the strings 'supernovas', 'supernovas', 'Supernovas', and 'SupernovaS', but not with the string 'supernova'. This is achieved with the use of a question mark /?/. A question mark makes the preceding character optional, i.e., zero or one occurrence of the previous character. The regular expression

/supernovas?/

specifies both 'supernova' and 'supernovas'. Often, we need to specify repeated occurrences of a character. The * operator, called the *Kleene* * (pronounced 'cleany star'), allow us to do this. The * operator specifies zero or more occurrences of a preceding character or regular expression. The regular expression /b*/ will match any string containing zero or more occurrences of 'b', i.e., it will match 'b', 'bb', or 'bbbb'. It will also match 'aaa', since that string contains zero occurrences of 'b'. To match a string containing one or more 'b's, the regular expression is /bb*/. The regular expression /bb*/ means a 'b' followed by zero or more 'b's. This will match with any of the strings 'b', 'bb', 'bbb', 'bbbb'. Similarly, the regular expression /[ab]*/ specifies 'zero or more "a"s or "b"s. This will match strings like 'aa', 'bb', or 'abab'. The kleene+ provides a shorter notation to specify one or more occurrences of a character. The regular

expression /a+/ means one or more occurrences of 'a'. Using *Kleene+*, we can specify a sequence of digits by the regular expression /[0–9]+/. Complex regular expressions can be built up from simpler ones by means of regular expression operators.

The caret (^) is also used as an anchor to specify a match at the beginning of a line. The dollar sign, $, is an anchor that is used to specify a match at the end of the line. ^ and $ are important to regexes. If you wish to search for a line containing only the phrase 'The nature.' and nothing else, you need to specify a regular expression for this search. The anchors ^ and $ are of great help in this case. The regular expression /^The nature\.$/ will search exactly for this line.

A number of special characters are also used to build regular expressions. One such character is the dot (.), called wildcard character, which matches any single character. The wildcard expression /./ matches any single character. The regular expression /.at/ matches with any of the string cat, bat, rat, gat, kat, mat, etc. It will also match the meaningless words such as nat, 4at, etc. Table 3.3 shows some of the special characters and their likely use.

Table 3.3 Some special characters

RE	Description
.	The dot matches any single character.
\n	Matches a new line character (or CR+LF combination).
\t	Matches a tab (ASCII 9).
\d	Matches a digit [0–9].
\D	Matches a non-digit.
\w	Matches an alphanumeric character.
\W	Matches a non-alphanumberic character.
\s	Matches a whitespace character.
\S	Matches a non-whitespace character.
\	Use \ to escape special characters. For example, \. matches a dot, * matches a * and \\ matches a backslash.

We can also use the wildcard symbol for counting characters. For instance /.....berry/ matches ten-letter strings that end in berry. This finds patterns like strawberry, sugarberry, and blackberry but fails to match with blueberry or hackberry.

Suppose you are searching a text for the presence of 'Tanveer' or 'Siddiqui'. You cannot use square brackets for this. You need a disjunction operator, shown by the pipe symbol(|). The pattern blackberry|blackberries matches either 'blackberry' or 'blackberries'. You might prefer to do this

matching by merely writing blackberry|ies. Unfortunately, this does not work. The pattern blackberry|ies matches with either 'blackberry' or 'ies'. This is because sequences take precedence over disjunction. In order to apply the disjunction operator to a specific pattern, we need to enclose it within parentheses. The parenthesis operator makes it possible to treat the enclosed pattern as a single character for the purposes of neighboring operators. Now, we will consider an example from real application.

Example 3.1 Suppose we need to check if a string is an email address or not. An email address consist of a non-empty sequence of characters followed by the 'at' symbol, @, followed by another non-empty sequence of characters ending with pattern like .xx, .xxx, .xxxx, etc. The regular expression for an email address is

^[A-Za-z0-9_\.-]$^{+}$+@[A-Za-z0-9_\.-]$^{+}$+[A-Za-z0-9_][A-Za-z0-9_]$

Table 3.4 shows the various parts of this 'rgex'.

Table 3.4 Parts of regular expression of Example 3.1

Pattern	Description
^[A-Za-z0-9_\.-]+	Match a positive number of acceptable characters at the start of the string.
@	Match the @ sign.
[A-Za-z0-9_\.-]+	Match any domain name, including a dot.
[A-Za-z0-9_][A-Za-z0-9_]$	Match two acceptable characters but not a dot. This ensures that the email address ends with .xx, .xxx, .xxxx, etc.

This example works for most cases. However, the specification is not based on any standard and may not be accurate enough to match all correct email addresses. It may accept non-working email addresses and reject working ones. Fine-tuning is required for accurate characterization.

A regular expression characterizes a particular kind of formal language, called a regular language. The language of regular expressions is similar to formulas of Boolean logic. Like logic formulas, regular expressions represent sets. Regular language is set of strings described by the regular expression. Regular languages may be encoded as finite state networks.

A regular expression may contain symbol pairs. For example, the regular expression /a:b/ represents a pair of string. The regular expression /a:b/ actually denotes a regular relation. A regular relation may be viewed as a mapping between two regular languages. The a:b relation is simply the cross product of the languages denoted by the expressions /a/ and /b/. To differentiate the two languages that are involved in a regular relation, we call the first one, the upper, and the second one, the lower

language, of the relation. Similarly, in the pair /a:b/, the first symbol, a, can be called the upper symbol and the second symbol, b, the lower symbol. The two components of a symbol pair are separated in our notation by a colon (:) without any whitespace before or after. To make the notation less cumbersome, we ignore the distinction between the language A and the identity relation that maps every string of A to itself. Therefore, we also write /a:a/ simply as /a/.

Regular expressions have clean, declarative semantics (Voutilainen 1996). Mathematically, they are equivalent to finite automata, both having the same expressive power. This makes it possible to encode regular languages using finite-state automata, leading to easier manipulation of context free and other complex languages. Similarly, regular relations can be represented using finite-state transducers. With this representation, it is possible to derive new regular languages and relations by applying regular operators, instead of re-writing the grammars.

3.3 FINITE-STATE AUTOMATA

In our childhood, each of us must have played some game that fits the following description:

Pieces are set up on a playing board; dice are thrown or a wheel is spun, and a number is generated at random. Based on the number appearing on the dice, the pieces on the board are rearranged specified by the rules of the game. Then, it is your opponent's turn; she also does the same thing, resulting in rearrangement of the pieces based on the number generated. There is no skill or choice involved. The entire game is based on the values of the random numbers.

Consider all possible positions of the pieces on the board and call them *states*. The state in which the game begins is termed the *initial state*, and the state corresponding to the winning positions is termed the *final state*. We begin with the initial state of the starting positions of the pieces on the board. The game then changes from one state to another based on the value of the random number. For each possible number, there is one and only one resulting state given the input of the number and the current state. This continues until one player wins and the game is over. This is called a final state.

Now consider a very simple machine with an input device, a processor, some memory, and an output device. The machine starts in the initial state. It checks the input and goes to next state, which is completely determined by the prior state and the input. If all goes well, the machine

reaches final state and terminates. If the machine gets an input for which the next state is not specified, and it gets stuck at a non-final state, we say the machine has failed or rejected the input.

A general model of this type of machine is called a finite automaton; 'finite' because the number of states and the alphabet of input symbols is finite; 'automaton' because the machine moves automatically, i.e., the change of state is completely governed by the input. This type of machine is more commonly called deterministic.

A finite automaton has the following properties:

1. A finite set of states, one of which is designated the *initial* or *start state*, and one or more of which are designated as the *final states.*
2. A finite alphabet set, Σ, consisting of input symbols.
3. A finite set of *transitions* that specify for *each* state and *each* symbol of the input alphabet, the state to which it next goes.

A finite automaton can be deterministic or non-deterministic. In a non-deterministic automaton, more than one transition out of a state is possible for the same input symbol.

Example 3.2 Suppose $\Sigma = \{a, b\}$, the set of states $= \{q_0, q_1, q_2, q_3, q_4\}$ with q_0 being the start state and q_4 the final state, we have the following rules of transition:

1. From state q_0 and with input a, go to state q_1.
2. From state q_1 and with input b, go to state q_2.
3. From state q_1 and with input c, go to state q_3.
4. From state q_2 and with input b, go to state q_4.
5. From state q_3 and with input b, go to state q_4.

This finite-state automaton is shown as a directed graph, called transition diagram, in Figure 3.1. The nodes in this diagram correspond to the states, and the arcs to transitions. The arcs are labelled with inputs. The final state is represented by a double circle. As seen in the figure, there is exactly one transition leading out of each state. Hence, this automaton is deterministic.

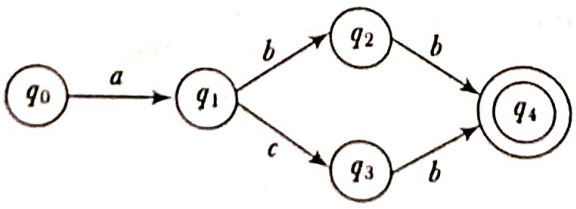

Figure 3.1 A deterministic finite-state automaton (DFA)

Finite-state automata have been used in a wide variety of areas, including linguistics, electrical engineering, computer science, mathematics, and logic. These are an important tool in computational linguistics and have been used as a mathematical device to implement regular expressions. Any regular expression can be represented by a finite automaton and the language of any finite automaton can be described by a regular expression. Both have the same expressive power. The following formal definitions of the two types of finite state automaton, namely, deterministic and non-deterministic finite automaton, are taken from Hopcroft and Ullman (1979).

A *deterministic finite-state automaton* (DFA) is defined as a 5-tuple $(Q, \Sigma, \delta, S, F)$, where Q is a set of *states*, Σ is an alphabet, S is the *start* state, $F \subseteq Q$ is a set of *final* states, and δ is a *transition function*. The transition function δ defines mapping from $Q \times \Sigma$ to Q. That is, for each state q and symbol a, there is at most one transition possible as shown in Figure 3.1.

Unlike DFA, the transition function of a *non-deterministic finite-state automaton* (NFA) maps $Q \times (\Sigma \cup \{\varepsilon\})$ to a subset of the power set of Q. That is, for each state, there can be more than one transition on a given symbol, each leading to a different state.

This is shown in Figure 3.2, where there are two possible transitions from state q_0 on input symbol a.

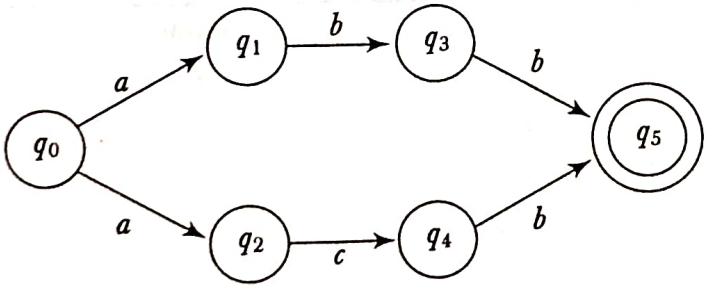

Figure 3.2 Non-deterministic finite-state automaton (NFA)

A *path* is a sequence of transitions beginning with the start state. A path leading to one of the final states is a *successful path*. The FSAs encode *regular* languages. The language that an FSA encodes is the set of strings that can be formed by concatenating the symbols along each successful path. Clearly, for automata with cycles, these sets are not finite.

We now examine what happens to various input strings that are presented to finite state automata. Consider the deterministic automaton described in Example 3.2 and the input, *ac*. We start with state q_0 and go to state q_1. The next input symbol is *c*, so we go to state q_3. No more input is left and we have not reached the final state, i.e., we have an unsuccessful end. Hence, the string *ac* is not recognized by the automaton.

This example illustrates how an FSA can be used to accept or recognize a string. The set of all strings that leave us in a final state is called the language accepted or defined by the FA. This means *ac* is not a word in the language defined by the automaton of Figure 3.1.

Now, consider the input acb. Again, we start with state q_0 and go to state q_1. The next input symbol is *c*, so we go to state q_3. The next input symbol is *b*, which leads to state q_4. No more input is left and we have reached to final state, i.e., this time we have a successful termination. Hence, the string *acb* is a word of the language defined by the automaton.

The language defined by this automaton can be described by the regular expression */abb|acb/*.

The example considered here is quite simple. Typically, the list of transition rules can be quite long. Listing all transition rules may be inconvenient, so often we represent an automaton as a *state-transition table*. The rows in this table represent states and the columns correspond to input. The entries in the table represent the transition corresponding to a given state-input pair. A ϕ entry indicates missing transition. This table contains all the information needed by FSA. The state transition table for the automaton considered in Example 3.2 is shown in Table 3.5.

Table 3.5 The state-transition table for the DFA shown in Figure 3.1

		Input		
State		***a***	***b***	***c***
start:	q_0	q_1	ϕ	ϕ
	q_1	ϕ	q_2	q_3
	q_2	ϕ	q_4	ϕ
	q_3	ϕ	q_4	ϕ
final:	q_4	ϕ	ϕ	ϕ

Now, consider a language consisting of all strings containing only *a*s and *b*s and ending with *baa*. We can specify this language by the regular expression */(a|b)*baa$/*. The NFA implementing this regular expression is shown in Figure 3.3.

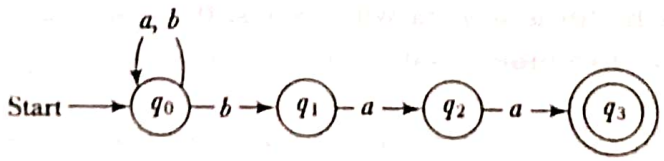

Figure 3.3 NFA for the regular expression /(a|b)*baa$/

Table 3.6 The state-transition table for the NFA shown in Figure 3.3

State		Input	
		a	*b*
start:	q_0	$\{q_0\}$	$\{q_0, q_1\}$
	q_1	$\{q_2\}$	ϕ
	q_2	$\{q_3\}$	ϕ
final:	q_3	ϕ	ϕ

Two automata that define the same language are said to be *equivalent*. An NFA can be converted to an equivalent DFA and vice versa. The equivalent DFA for the NFA shown in Figure 3.3 is shown in Figure 3.4.

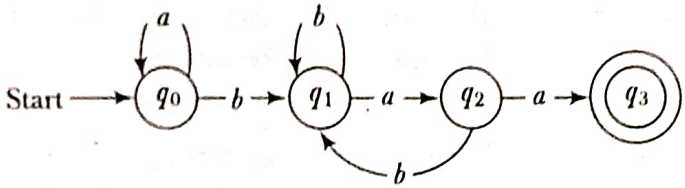

Figure 3.4 Equivalent DFA for NFA in Figure 3.3

3.4 MORPHOLOGICAL PARSING

Morphology is a sub-discipline of linguistics. It studies word structure and the formation of words from smaller units (morphemes). The goal of morphological parsing is to discover the morphemes that build a given word. Morphemes are the smallest meaning-bearing units in a language. For example, the word 'bread' consists of a single morpheme and 'eggs' consist of two: the morpheme egg and the morpheme -s. A morphological parser should be able to tell us that the word 'eggs' is the plural form of the noun stem 'egg'.

There are two broad classes of morphemes: stems and affixes. The stem is the main morpheme, i.e., the morpheme that contains the central meaning. Affixes modify the meaning given by the stem. Affixes are divided into prefix, suffix, infix, and circumfix. Prefixes are morphemes which appear before a stem, and suffixes are morphemes applied to the end of the stem. Circumfixes are morphemes that may be applied to either end of the stem while infixes are morphemes that appear inside a stem. Prefixes and suffixes are quite common in Urdu, Hindi, and English. For example, the Urdu word, بےوقت (*bewaqt*), meaning untimely, is composed of the stem, *waqt*, and the prefix, *be-*, گھوڑوں (*ghodhon*) is composed of the stem, گھوڑا (*ghodha*) and the suffix, وں (*on*). Also, the word, ضرورت مند (*zarooratmand*), is composed of the stem, ضرورت

(*zaroorat*), and the suffix, مند (*mand*). Similarly, the English word, unhappy, is composed of the stem, happy, and the prefix, un-. The English word birds, is composed of the stem, bird, and the suffix, -s. Likewise, the Hindi word, शीतलता is composed of a stem शीतल and the suffix – ता.

Telgu word గుర్రములు (*gurramulu*–plural form of *gurramu*, meaning horse) is composed of the stem గుర్రము (*gurramu*) and suffix -లు (*lu*). Some commonly used suffixes in plural forms of Telugu nouns are: లు (*lu*), ళ్లు (*llu*), ల్లు (*dlu*). Here is a list of singular and plural forms of a few Telugu words:

Singular	Meaning	Plural
పిల్లి (*pilli*)	Cat	పిల్లులు (*pillulu*)
పడుచు (*paduchu*)	Women	పడుచులు (*paduchulu*)
ఆడది (*aadadi*)	Women	ఆడవాండ్లు (*aadawandlu** or *aadawalu*)
మగవాడు (*magavadu*)	Man	మగవాండ్లు (*magavandlu* or *magavallu*)
పురుషుడు (*purushudu*)	Man	పురుషులు (*purushulu*)
చెవి (*chevi*)	Ear	చెవులు (*chevulu*)
ఇల్లు (*illu*)	House	ఇల్లులు (*illulu* or *illlu*)

*Another plural form of *aadadi* is *adawaru*, which is used to show respect.

There are three main ways of word formation: inflection, derivation, and compounding. In inflection, a root word is combined with a grammatical morpheme to yield a word of the same class as the original stem. Derivation combines a word stem with a grammatical morpheme to yield a word belonging to a different class, e.g., formation of the noun 'computation' from the verb 'compute'. The formation of a noun from a verb or adjective is called nominalization. Compounding is the process of merging two or more words to form a new word. For example, personal computer, desktop, overlook. Morphological analysis and generation deal with inflection, derivation and compounding process in word formation.

New words are continually forming a natural language. Many of these are morphologically related to known words. Understanding morphology is therefore important to understand the syntactic and semantic properties of new words. Morphological analysis and generation are essential to many NLP applications ranging from spelling corrections to machine translations. In parsing, e.g., it helps to know the agreement features of words. In information retrieval, morphological analysis helps identify the presence of a query word in a document in spite of different morphological variants.

Parsing, in general, means taking an input and producing some sort of structures for it. In NLP, this structure might be morphological, syntactic, semantic, or pragmatic. Morphological parsing takes as input the inflected

surface form of each word in a text. As output, it produces the parsed form consisting of a canonical form (or *lemma*) of the word and a set of tags showing its syntactical category and morphological characteristics, e.g., possible part of speech and/or inflectional properties (gender, number, person, tense, etc.). Morphological generation is the inverse of this process. Both analysis and generation rely on two sources of information: a dictionary of the valid lemmas of the language and a set of inflection paradigms.

A morphological parser uses following information sources:

1. **Lexicon**
   A lexicon lists stems and affixes together with basic information about them.
2. **Morphotactics**
   There exists certain ordering among the morphemes that constitute a word. They cannot be arranged arbitrarily. For example, rest-less-ness is a valid word in English but not rest-ness-less. Morphotactics deals with the ordering of morphemes. It describes the way morphemes are arranged or touch each other.
3. **Orthographic rules**
   These are spelling rules that specify the changes that occur when two given morphemes combine. For example the $y \rightarrow ier$ spelling rule changes '*easy*' to '*easier*' and not to '*easyer*'.

Morphological analysis can be avoided if an exhaustive lexicon is available that lists features for all the word-forms of all the roots. Given a word, we simply consult the lexicon to get its feature values. For example, suppose an exhaustive lexicon for Hindi contains the following entries for the Hindi root-word *ghodhaa*.

Table 3.7 A sample lexicon entry

Word form	Category	Root	Gender	Number	Person
Ghodhaa	noun	GhoDaa	masculine	singular	3rd
Ghodhii	-do-	-do-	feminine	-do-	-do-
Ghodhon	-do-	-do-	masculine	plural	-do-
Ghodhe	-do-	-do-	-do-	-do-	-do-

Given a word, say *ghodhon*, we can look up the lexicon to get its feature values.

However, this approach has several limitations. First, it puts a heavy demand on memory. We have to list every form of the word, which results in a large number of, often redundant, entries in the lexicon.

Second, an exhaustive lexicon fails to show the relationship betwe different roots having similar word-forms. That means the approach fa to capture linguistic generalization, which is essential to develop a syste capable of understanding unknown words.

Third, for morphologically complex languages, like Turkish, the numb of possible word-forms may be theoretically infinite. It is not practical t list all possible word-forms in these languages.

These limitations explain why morphological parsing is necessary. Th complexity of the morphological analysis varies widely among the world' languages, and is quite high even in relatively simple cases, such a English.

The simplest morphological systems are stemmers that collaps morphological variations of a given word (word-forms) to one lemma o stem. They do not require a lexicon. Stemmers have been especially used in information retrieval. Two widely used stemming algorithms have been developed by Lovins (1968) and Porter (1980). Stemmers do not use a lexicon; instead, they make use of rewrite rules of the form:

ier → *y* (e.g., earlier → early)

ing → ε (e.g., playing → play)

Stemming algorithms work in two steps:

(i) Suffix removal: This step removes predefined endings from words.

(ii) Recoding: This step adds predefined endings to the output of the first step.

These two steps can be performed sequentially as in Lovins's stemmer or simultaneously as in Porter's stemmer. For example, Porter's stemmer makes use of the following transformation rule:

ational → *ate*

to transform word such as 'rotational' into 'rotate'.

It is difficult to use stemming with morphologically rich languages. Even in English, stemmers are not perfect. Krovitz (1993) pointed out errors of omissions and commissions in the Porter algorithm, such as transformation of the word 'organization' into 'organ' and 'noise' into 'noisy'. Another problem with Porter's algorithm is that it reduces only suffixes; prefixes and compounds are not reduced.

A more efficient *two-level morphological model*, first proposed by Koskenniemi (1983), can be used for highly inflected languages. In this model, a word is represented as a correspondence between its lexical level form and its surface level form. The surface level represents the actual spelling of the word while the lexical level represents the concatenation of its constituent morphemes. Morphological parsing is viewed as a mapping from the surface level into morpheme and feature sequences on the lexical level.

For example, the surface form 'playing' is represented in the lexical form as play + V + PP as shown in Figure 3.5. The lexical form consists of the stem 'play' followed by the morphological information +V +PP, which tells us that 'playing' is the present participle form of the verb.

Surface level	p	l	a	y	i	n	g

Lexical level	p	l	a	y	+V	PP

Figure 3.5 Surface and lexical forms of a word

Similarly, the surface form 'books' is represented in the lexical form as 'book + N + PL', where the first component is the stem and the second component (N + PL) is the morphological information, which tells us that the surface level form is a plural noun.

This model is usually implemented with a kind of finite-state automata, called *finite-state transducer* (FST). A transducer maps a set of symbols to another. A finite state transducer does this through a finite state automaton. An FST can be thought of as a two-state automaton, which recognizes or generates a pair of strings. FST passes over the input string by consuming the input symbols on the tape it traverses and consists it to the output string in the form of symbols. Formally, an FST has been defined by Hopcroft and Ullman (1979) as follows:

A finite-state transducer is a 6-tuple $(\Sigma_1, \Sigma_2, Q, \delta, S, F)$, where θ is set of states, S is the initial state, and $F \subseteq Q$ is a set of final states, Σ_1 is *input* alphabet, Σ_2 is *output* alphabet, and δ is a function mapping $Q \times (\Sigma_1 \cup \{\varepsilon\}) \times (\Sigma_2 \cup \{\varepsilon\})$ to a subset of the power set of Q.

Transducers can be seen as automata with transitions labelled with symbols from $\Sigma_1 \times \Sigma_2$, where Σ_1 and Σ_2 are the alphabets of input and output respectively. Thus, an FST is similar to an NFA except in that transitions are made on strings rather than on symbols and, in addition, they have outputs.

Figure 3.6 shows a simple transducer that accepts two input strings, hot and cat, and maps them onto cot and bat respectively. It is a common practice to represent a pair like *a*:*a* by a single letter.

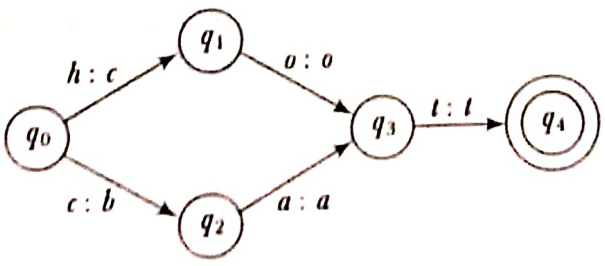

Figure 3.6 Finite-state transducer

Just as FSAs encode regular languages, FSTs encode regular relations. Regular relation is the relation between regular languages. The regular language encoded on the upper side of an FST is called upper language, and the one on the lower side is termed lower language. If T is a transducer, and s is a string, then we use $T(s)$ to represent the set of strings encoded by T such that the pair (s, t) is in the relation.

The FSTs are closed under union concatenation, composition, and Kleene closure. However, in general, they are not closed under intersection and complementation.

With this introduction, we can now implement the two-level morphology using FST. To get from the surface form of a word to its morphological analysis, we proceed in two steps as illustrated in Figure 3.7.

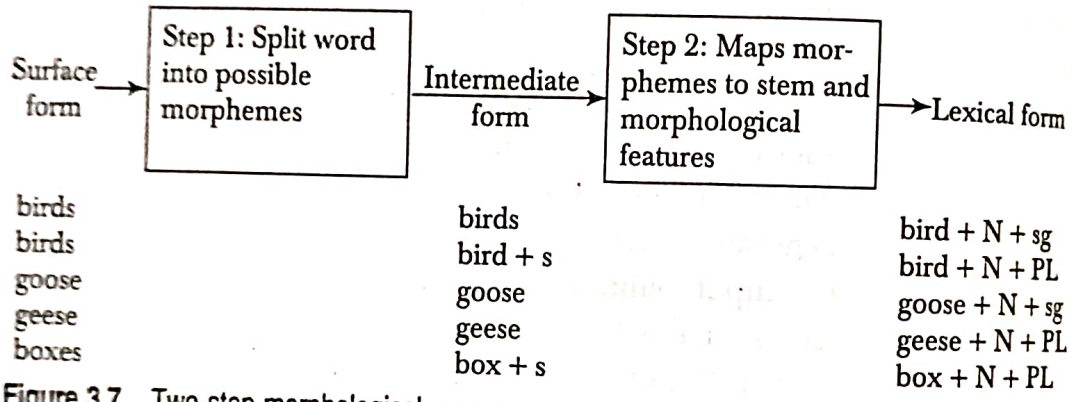

Figure 3.7 Two-step morphological parser

First, we split the words up into its possible components. For example, we make *bird* + *s* out of *birds*, where + indicates morpheme boundaries. In this step, we also consider spelling rules. Thus, there are two possible ways of splitting up *boxes*, namely *boxe* + *s* and *box* + *s*. The first one assumes that *boxe* is the stem and *s* the suffix, while the second assumes that *box* the stem is and that *e* has been introduced due to the spelling rule. The output of this step is a concatenation of morphemes, i.e., stems and affixes. There can be more than one representation for a given word. A transducer that does the mapping (translation) required by this step for the surface form 'lesser' might look like Figure 3.8. This FST represents the information that the comparative form of the adjective *less* is *lesser*, ε here is the empty string. The automaton is inherently bi-directional: the same transducer can be used for analysis (surface input, 'upward' application) or for generation (lexical input, 'downward' application).

Figure 3.8 A simple FST

In the second step, we use a lexicon to look up categories of the stems and meaning of the affixes. So, *bird* + *s* will be mapped to *bird*+ *N*+ *PL*, and *box* + *s* to *box*+ *N* + *PL*. We also find out now that *boxe* is not a legal stem. This tells us that splitting *boxes* into *boxe* + *s* is an incorrect way of splitting *boxes*, and should therefore be discarded. This may not be the case always. We have words like *spouses* or *parses* where splitting the word into *spouse* + *s* or *parse* + *s* is correct. Orthographic rules are used to handle these spelling variations. For instance, one of the spelling rules says- add e after -s, -z, -x, -ch, -sh before the s (e.g., dish→dishes, box→boxes).

Each of these steps can be implemented with the help of a transducer. Thus, we need to build two transducers: one that maps the surface form to the intermediate form and another that maps the intermediate form to the lexical form.

We now develop an FST-based morphological parser for singular and plural nouns in English. The plural form of regular nouns usually end with -s or -es. However, a word ending in 's' need not necessarily be the plural form of a word. There are a number of singular words ending in s, e.g., miss and ass. One of the required translations is the deletion of the 'e' when introducing a morpheme boundary. This deletion is usually required for words ending in xes, ses, zes (e.g., suffixes and boxes). The transducer in Figure 3.9 does this. Figure 3.10 shows the possible sequences of states that the transducer undergoes, given the surface forms *birds* and *boxes* as input.

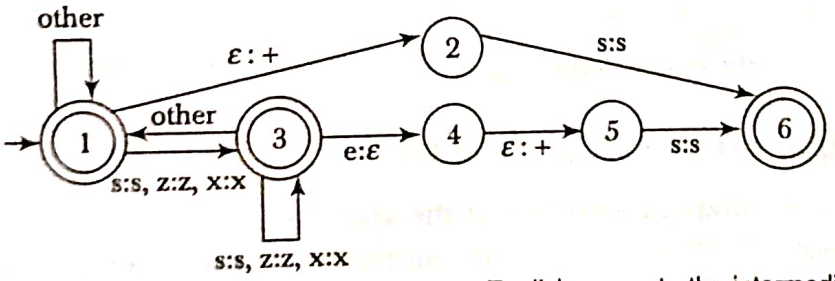

Figure 3.9 A simplified FST, mapping English nouns to the intermediate form

The next step is to develop a transducer that does the mapping from the intermediate level to the lexical level. The input to transducer has one of the following forms:

- Regular noun stem, e.g., *bird*, *cat*
- Regular noun stem + s, e.g., *bird* + *s*
- Singular irregular noun stem, e.g., *goose*
- Plural irregular noun stem, e.g., *geese*

In the first case, the transducer has to map all symbols of the stem to themselves and then output *N* and *sg* (Figure 3.7). In the second case, it

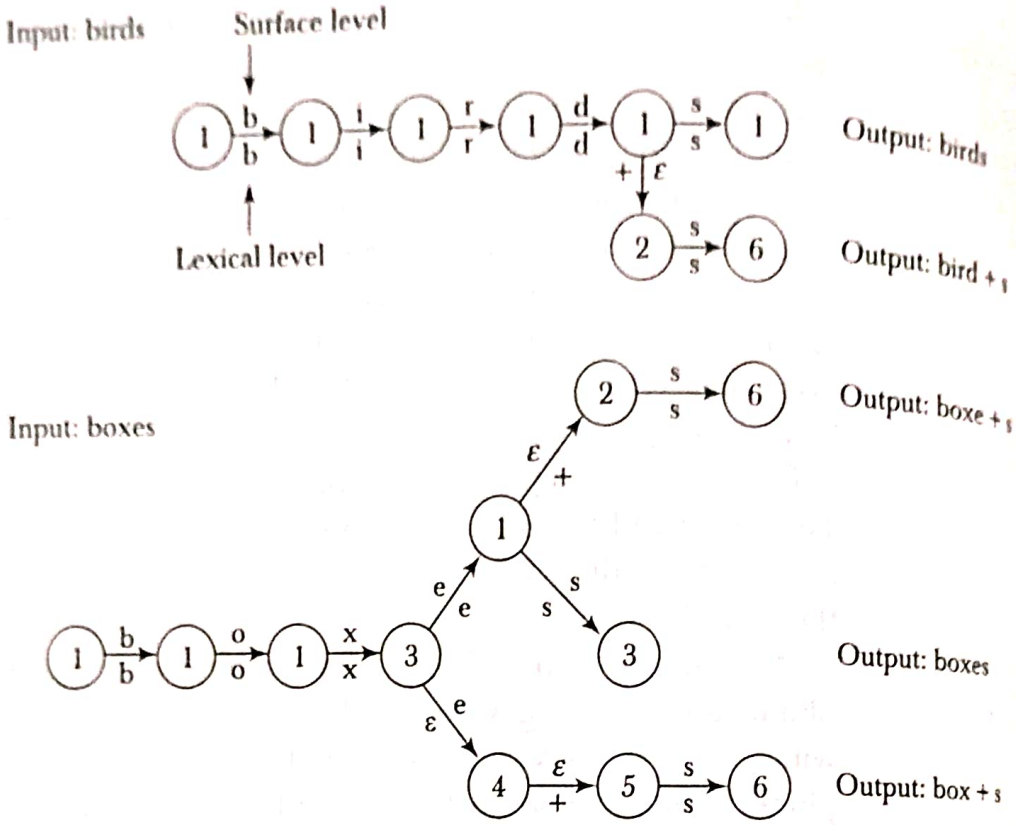

Figure 3.10 Possible sequences of states

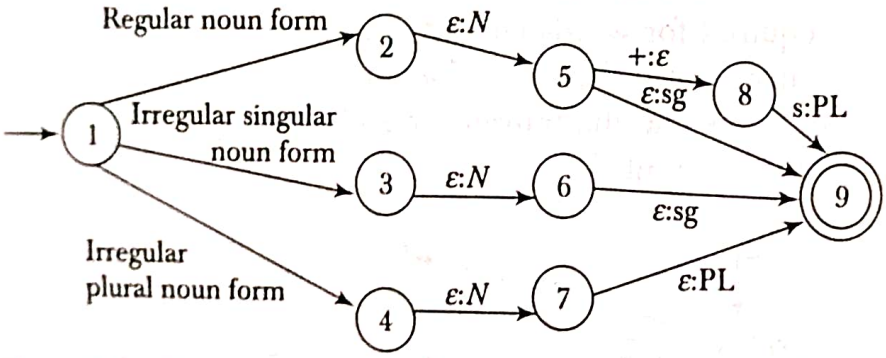

Figure 3.11 Transducer for Step 2

has to map all symbols of the stem to themselves, but then output *N* an replaces *PL* with *s*. In the third case, it has to do the same as in the fi case. Finally, in the fourth case, the transducer has to map the irregul plural noun stem to the corresponding singular stem (e.g., *geese* to *goo* and then it should add *N* and *PL*. The general structure of this transduc looks like Figure 3.11.

The mapping from State 1 to State 2, 3, or 4 is carried out with t help of a transducer encoding a lexicon. The transducer implementi the lexicon maps the individual regular and irregular noun stems to the correct noun stem, replacing labels like regular noun form, etc. Th lexicon maps the surface form *geese*, which is an irregular noun, to correct stem *goose* in the following way:

g:g e:o e:o s:s e:e

Mapping for the regular surface form of *bird* is b:b i:i r:r d:d. Representing pairs like *a:a* with a single letter, these two representations are reduced to g e:o e:o s e and b i r d respectively.

Composing this transducer with the previous one, we get a single two-level transducer with one input tape and one output tape. This maps plural nouns into the stem plus the morphological marker + pl and singular nouns into the stem plus the morpheme + sg. Thus a surface word form *birds* will be mapped to bird + N + pl as follows.

b:b i:i r:r d:d + ε:N + s:pl

Each letter maps to itself, while ε maps to morphological feature +N, and s maps to morphological feature pl. Figure 3.12 shows the resulting composed transducer.

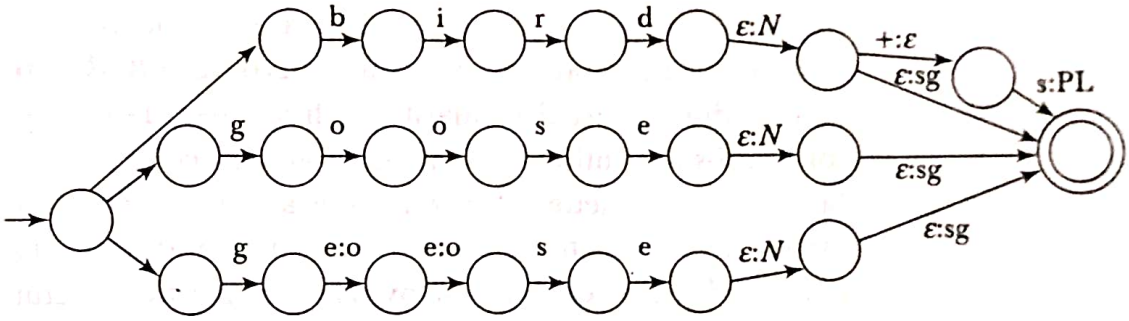

Figure 3.12 A transducer mapping nouns to their stem and morphological features

The power of the transducer lies in the fact that the same transducer can be used for analysis and generation. That is, we can run it in the downward direction (input: surface form and output: lexical form) or in the upward direction.

3.5 SPELLING ERROR DETECTION AND CORRECTION

In computer-based information systems, errors of typing and spelling constitute a very common source of variation between strings. These errors have been widely investigated. All investigations agree that single character omission, insertion, substitution, and reversal are the most common typing mistakes. In an early investigation, Damearu (1964) reported that over 80% of the typing errors were single-error misspellings:

(1) substitution of a single letter,
(2) omission of a single letter,
(3) insertion of a single letter, and
(4) transposition of two adjacent letters.